Principles & Practices Document: AI Model Results Analysis Platform (AMRAP)

Course: CS5013 PAI | Pre-Evaluation Deliverable
Name: Ferdeno | **Roll No:** EE26S062

Date: 08 October 2026
Repository: github.com/FerdenoIITM/AI-Model-Results-Analysis-Platform (branch `dev`)

1. Project at a Glance

AMRAP stores AI model evaluation results, lets users filter, sort and page through them, and answers natural-language questions about them. External clients submit results (raw JSON plus metadata such as category and six metrics) to an ingest API. The JSON is written to disk, the metadata to PostgreSQL, and a background worker embeds the content into **pgvector** so a local **Ollama** LLM can answer questions with retrieval-augmented generation (RAG). The stack is Spring Boot 3 (Java 21, multi-module Maven), React 18 + Vite, PostgreSQL 16 and Docker Compose, with role-based access (superadmin, admin, member) secured by JWT. The project was built over three weeks (20 Sep to 8 Oct 2026) with an AI coding assistant. The practices below are how I kept that collaboration correct, traceable and maintainable.

2. Prompt Engineering

My prompts evolved from open questions ("how should I organize a website?") into **specifications** that a coding agent can execute and that I can check. The principles I apply, each with a real example from `Prompt.md` (samples in Appendix A):

Principle	How I apply it	Example from my prompts
Ground in the real code	Ask the agent to inspect files before proposing anything; name exact paths and terminal line ranges.	"First inspect the actual folders and files. Do not assume the architecture." / "In terminals/4.txt lines 98-105, why is the container restarting?"
Decompose incrementally	One concern per prompt, in dependency order: schema, then API, then UI, then tests.	"Do not jump directly into the complete application. For now, focus only on the database design and schema."
Structured template (C-O-O-S-T-A-R)	Context, Objective, Output, Style, Tone, Audience, Response constraints. Each section removes a class of ambiguity.	Superadmin-only result submission and chat follow-up history prompts (Appendix A.4).
Negative constraints and scope	List what must *not* change, and ask for the smallest possible change, to stop unrelated refactors.	"Do not refactor unrelated code." "Make the smallest possible change required to persist the existing chat state."
Explicit acceptance criteria	End with a "Done when" list I can verify by hand or with tests.	"Done when: ... Admin and member do not see Submit result, and their JWT gets 403 from POST /api/results."
Explain before fixing	For bugs and unfamiliar concepts, first ask for the cause, then restate my understanding to confirm it.	"First explain the problem - do not jump to the fix." / "So you are saying ... Is that correct?"
Plan, then implement	Large features start in plan mode; the agent asks clarifying questions and I approve a written plan before code changes.	"Give me a plan for implementing this using RAG." The RAG feature and the test suite were both planned and approved before any code.

3. Debugging

I follow the same loop for every issue, whether it comes from a terminal, the browser or a failing test:

```
1 Reproduce -> 2 Collect evidence (exact log lines, status codes) -> 3 Explain the cause before
any fix -> 4 Confirm the hypothesis -> 5 Apply the smallest fix -> 6 Verify, then add a
regression test
```

Issue	Evidence and diagnosis	Fix and verification
API container in a restart loop	Read the container logs by line range: the API could not reach PostgreSQL. Asked for an explanation first: Postgres ran on the host and the API in Docker, so they sat in different network namespaces and Postgres listened only on localhost.	Point the API at `host.docker.internal` (mapped to `host-gateway` in Compose) and let host Postgres accept connections from Docker; confirmed by re-reading the logs until the container stayed healthy.
Wrong status codes and sort order found by the new tests	Integration tests showed 403 instead of 401 without a token, 500 for a malformed UUID or unknown route, nulls first in descending sort, and 500 for two simultaneous creates with the same email.	Minimal fixes: a JSON 401 entry point, exception-to-status mapping, NULLS LAST with an id tie-breaker, unique-violation to 409. Each fix is now held by a test.
Flaky end-to-end pagination test	Page 2 "overlapped" page 1. Traced it to two causes: the page renders once with the new page number before loading starts, and iteration IDs are second-resolution timestamps, so they are not unique.	Wait for the actual `GET /api/results` response; identify rows by model+version. Repeated the read-only specs 3 times (42 runs, all stable).

Two habits make this reliable: I quote exact evidence (file and line range) instead of describing symptoms, and I check a fix catches the bug by breaking the code on purpose (for example, disabling the Clear Filters reset made its test fail).

4. Version Control with Git (Code and Prompts)

- **Branches and remote:** all development happens on `dev` (28 commits, pushed to GitHub); `main` is kept for a stable, evaluated version and currently holds only the initial commit.
- **Feature-sized commits with descriptive messages** that say what changed and why, for example *"Add column sorting functionality to results table ... Ensure sorting uses numeric values and does not affect existing column sorting"*. Each commit maps to one prompt or feature, so the history doubles as a changelog (Appendix B).
- **Prompts are versioned with the code.** `Prompt.md` (about 2,000 lines, 9 dated sections) records every prompt I gave the assistant. It changed in 14 commits, usually *the same commit as the code it produced* (for example, the filtering feature commit added both the code and its 680-line prompt). `git log -p Prompt.md` shows how my prompting evolved, and `git blame` links any feature to the prompt that requested it. I refactored the file once (5 Oct) to remove duplicated entries.
- **Secrets stay out of git:** `.env` is ignored and `.env.example` documents every variable with safe placeholders; build output, `node_modules`, coverage and test reports are ignored too.
- **Planned next:** short-lived feature branches merged into `dev` by pull request, and a tagged release merged to `main` for each evaluation.

5. Testing, Refactoring and Documentation

Testing. One command, `make test`, runs six suites and prints a pass/fail table with coverage, returning a non-zero exit code on any failure. Tests cover failure paths, not only happy paths: invalid, empty and null input, boundary values (0, 100, 100.0001), wrong types, duplicates, 10,000-row datasets, no-match queries, network and database failures, unauthorized access and concurrency. Only the LLM is stubbed; everything else runs against real PostgreSQL + pgvector and a real HTTP API.

Suite	Tools	Tests	What it verifies
Frontend components	Vitest, RTL, MSW	198	Loading/empty/error states, filters, Clear Filters, sorting, pagination, modals, nav highlighting.
Backend unit	JUnit 5, Mockito	203	Filter parsing edge cases, services, JWT and auth filters.
Backend integration	Testcontainers	117	Repository queries, every endpoint, security matrix, DB failure, concurrent writes.
Database (SQL)	psql assertions	223	Schema, constraints, foreign keys, trigger, index usage, sort/paging on 10k rows.

API black-box	Vitest + fetch	159	Status codes, response shapes, invalid filters, CORS, chat 502 without an LLM.
End-to-end	Playwright	22	Model, Category, Confidence >= 95 flow, sort, clear, submit modal at 1280x600, users, access.
Total (all passing)		**922**	**Line coverage: 99.2% frontend, 96.9% backend (unit + integration).**

API and end-to-end tests run against an isolated Docker stack (own database on tmpfs, test-only accounts, separate ports), so development data is never touched. Application code changed only where a test proved a defect, and each fix is recorded.

Refactoring. I refactor in small, behaviour-preserving steps, with tests as the safety net: simplified result storage to `<uuid>.json`; removed the unused `failed_login_attempts`/`locked_until` columns together with their code; consolidated error handling into one exception handler; moved null ordering into the repository query. The layout separates concerns: Maven modules for domain, persistence, application, web and security, and feature folders in React.

Documentation. Docs change in the same commit as the code: `README.md` (features, setup, LAN sharing, a Testing section), `docs/architecture.md`, `docs/data-model.md`, `docs/api.md`, and the OpenAPI contract `packages/openapi/openapi.yaml`, which I treat as the source of truth for the API. Prompts that change behaviour ask for these updates explicitly (for example, "Update packages/openapi/openapi.yaml, docs/api.md, and the README role table").

Appendix A: Sample Prompts (excerpts from Prompt.md)

A.1 Grounding and negative constraints (21 Sep, project structure)

```
Analyze the existing project at: [repository root]
First inspect the actual folders and files. Do not assume the architecture.
I want to establish a clean, industry-standard, scalable template for this project. [...]
Negative constraints. Do NOT: blindly restructure existing code; invent technologies; create
unnecessary folders; over-engineer; introduce microservices without a real need; put business logic
directly in controllers; put secrets/API keys in Git; create empty "enterprise-looking" folders.
Prefer simple, practical, maintainable architecture over maximum complexity.
```

Principles: grounding in the real repository, explicit negative constraints, preference for simplicity.

A.2 Incremental decomposition (24 Sep, database first)

```
I am building a web application with a role-based authentication and authorization system. I want to
design and implement the system step by step, starting with the PostgreSQL database schema.
Do not jump directly into the complete application implementation. For now, focus only on the database
design and schema. [...]
Important Constraints: Never store plaintext passwords. Do not rely only on frontend/UI restrictions
for authorization.
Clearly distinguish authentication (who the user is) from authorization (what the user is allowed to
do).
If any requirement is ambiguous, explain the options and recommend a suitable approach.
```

Principles: one layer at a time, security requirements stated up front, the agent asked to surface ambiguity.

A.3 Debugging: explain first, then confirm (24 Sep, API restart loop)

```
In terminals/4.txt lines 98-105, why is the container restarting, and where are the log files I can
read to find out why?
First explain the problem - do not jump to the fix. Why does Postgres need to listen on all addresses
here? State the issue clearly.
So you are saying Postgres is running on the host and the API is running inside Docker - they are in
different network namespaces and cannot talk to each other by default. Is that correct?
What is the fix for this? How do I let the API container reach host Postgres?
```

Principles: exact evidence (line ranges), diagnosis before fix, restating understanding to confirm the hypothesis.

A.4 Structured C-O-O-S-T-A-R specification (3 Oct, abridged)

```
C - Context: You are changing the existing AMRAP repo. Inspect the real code before editing. Relevant
pieces: ResultsPage.tsx, client.ts (createResult), ResultController.java POST /api/results,
ApiKeyAuthenticationFilter.java (header X-Ingest-Key) [...]
O - Objective: Make website result submission superadmin-only, and keep external machine ingest
working. [...]
O - Output: The smallest code change that enforces the rule in both the React page and the Java ingest
endpoint. Matching updates to the OpenAPI file and the two docs [...] A short summary of files changed
and how to verify.
S - Style: Precise and local. Follow the patterns already in ResultsPage (see canReindex) and
ResultController (see the reindex role check).
T - Tone: Direct and implementation-focused. No architecture essay.
A - Audience: A coding agent that will edit this repository and must not invent a new ingest API.
R - Response constraints: Do not: remove X-Ingest-Key authentication; block admin or member from GET
/api/results, viewing JSON, or using chat; put secrets in git; refactor unrelated pages [...]
Done when: Admin and member do not see Submit result, and their JWT gets 403 from POST /api/results.
POST /api/results with X-Ingest-Key [...] still creates a result. OpenAPI, docs/api.md, and the README
role table describe that split.
```

Principles: full template, reuse of existing patterns, documentation updates required, verifiable "Done when".

A.5 Minimal change with a "Do NOT" list (3 Oct, chat persistence, abridged)

```
Modify only the chat tab's state/memory persistence behavior.
Required behavior: Preserve the complete current conversation when switching from the Chat tab to
another tab such as Results. [...]
Negative requirements - Do NOT: change the existing Chat UI design; modify the existing RAG pipeline;
modify API endpoints or backend logic unless absolutely required; refactor unrelated code;
reload/recreate the Chat widget when switching tabs.
Important constraint: Make the smallest possible change required to persist the existing chat state
across tab switches.
```

Principles: tightly scoped objective, explicit exclusions, expected result stated as a flow.

A.6 Quality requirements for testing (8 Oct, abridged)

> Implement a complete automated test suite for the entire application covering Frontend, Backend, API, and Database.
> Use the existing project structure and technology stack. Do not change application functionality just to make tests pass.
> Do not test only happy paths. For every important feature include: valid, invalid, empty and null input; boundary values; incorrect data types; duplicate data; large datasets; no matching results; API, database and network failures; unauthorized/forbidden access; concurrent operations. [...]
> Do not create fake tests that only mock everything. Use real database/API integration tests where appropriate.
> make test runs the complete test suite and returns: clear test results; passed/failed count; coverage where supported; non-zero exit code when any required test fails.

Principles: measurable quality bar, anti-patterns named explicitly, a single verifiable entry point.

Appendix B: Commit History Excerpt (`git log --oneline`, branch dev)

```
7baf796  Project Structure Scaffold
8ec26ca  initial version of authentication
812e320  Set up hot-reload development environment with Docker; added docker-compose.dev.yml, updated
Makefile ...
cc5fe68  Integrate Ollama for RAG processing: added support for result ingestion and chat functionality
...
bf9d7f5  Implement chat state persistence across tab switches: modified ChatPage to retain conversation
history ...
9f4541a  Update result ingestion process: modify API endpoint to require `X-Ingest-Key` or superadmin
JWT ...
fb68e66  Enhance chat functionality: implement history tracking for follow-up questions ...
c0d62f1  Refactor Prompt.md: streamline content by removing redundant date entries and clarifying
instructions ...
e095408  Enhance AI Model Results Analysis Platform: extend result structure to include new fields for
category ...
b6ce072  Refactor result ingestion process: update file storage structure to save raw JSON directly ...
f5f4e6c  Add column sorting functionality to results table: implement sorting for new numeric metrics
...
33a135c  Add clear filters functionality to Results page: implement a button to reset all applied
filters ...
90840db  Update Makefile and README for testing enhancements: add new test commands to Makefile ...
```